

A Technical Introduction to Recurrent neural network

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

Section 1

Gated recurrent unit (GRU), introduced in 2014, was designed as a simplification of LSTM. They are used in the full form and several further simplified variants. They have fewer parameters than LSTM, as they lack an output gate. Their performance on polyphonic music modeling and speech signal modeling was found to be similar to that of long short-term memory. There does not appear to be particular performance difference between LSTM and GRU.

Section 2

The two output sequences are then concatenated to give the total output: $((y_0, y_0'), (y_1, y_1'), \dots, (y_N, y_N'))$ $\{ \displaystyle ((y_{\{0\}}, y_{\{0'\}}), (y_{\{1\}}, y_{\{1'\}}), \dots, (y_{\{N\}}, y_{\{N'\}})) \}$. Bidirectional RNN allows the model to process a token both in the context of what came before it and what came after it. By stacking multiple bidirectional RNNs together, the model can process a token increasingly contextually. The ELMo model (2018) is a stacked bidirectional LSTM which takes character-level as inputs and produces word-level embeddings.

Section 3

PixelRNN An RNN may process data with more than one dimension. PixelRNN processes two-dimensional data, with many possible directions. For example, the row-by-row direction processes an $n \times n$ $\{ \displaystyle n \times n \}$ grid of vectors $x_{i,j}$ $\{ \displaystyle x_{\{i,j\}} \}$ in the following order: $x_{1,1}, x_{1,2}, \dots, x_{1,n}, x_{2,1}, x_{2,2}, \dots, x_{2,n}, \dots, x_{n,1}, x_{n,2}, \dots, x_{n,n}$ $\{ \displaystyle x_{\{1,1\}}, x_{\{1,2\}}, \dots, x_{\{1,n\}}, x_{\{2,1\}}, x_{\{2,2\}}, \dots, x_{\{2,n\}}, \dots, x_{\{n,n\}} \}$. The diagonal BiLSTM uses two LSTMs to process the same grid. One processes it from the top-left corner to the bottom-right, such that it processes $x_{i,j}$ $\{ \displaystyle x_{\{i,j\}} \}$ depending on its hidden state and cell state on the top and the left side: $h_{i-1,j}, c_{i-1,j}$ $\{ \displaystyle h_{\{i-1,j\}}, c_{\{i-1,j\}} \}$ and $h_{i,j-1}, c_{i,j-1}$ $\{ \displaystyle h_{\{i,j-1\}}, c_{\{i,j-1\}} \}$. The other processes it from the top-right corner to the bottom-left.

Section 4

Two RNNs can be run front-to-back in an encoder-decoder configuration. The encoder RNN processes an input sequence into a sequence of hidden vectors, and the decoder RNN processes the sequence of hidden vectors to

an output sequence, with an optional attention mechanism. This was used to construct state of the art neural machine translators during the 2014–2017 period. This was an instrumental step towards the development of transformers.

Section 5

In artificial neural networks, recurrent neural networks (RNNs) are designed for processing sequential data, such as text, speech, and time series, where the order of elements is important. Unlike feedforward neural networks, which process inputs independently, RNNs utilize recurrent connections, where the output of a neuron at one time step is fed back as input to the network at the next time step. This enables RNNs to capture temporal dependencies and patterns within sequences. The fundamental building block of RNN is the recurrent unit, which maintains a hidden state—a form of memory that is updated at each time step based on the current input and the previous hidden state. This feedback mechanism allows the network to learn from past inputs and incorporate that knowledge into its current processing. RNNs have been successfully applied to tasks such as unsegmented, connected handwriting recognition, speech recognition, natural language processing, and neural machine translation. However, traditional RNNs suffer from the vanishing gradient problem, which limits their ability to learn long-range dependencies. This issue was addressed by the development of the long short-term memory (LSTM) architecture in 1997, making it the standard RNN variant for handling long-term dependencies. Later, gated recurrent units (GRUs) were introduced as a more computationally efficient alternative. In recent years, transformers, which rely on self-attention mechanisms instead of recurrence, have become the dominant architecture for many sequence-processing tasks, particularly in natural language processing, due to their superior handling of long-range dependencies and greater parallelizability. Nevertheless, RNNs remain relevant for applications where computational efficiency, real-time processing, or the inherent sequential nature of data is crucial.

Section 6

Neural history compressor The neural history compressor is an unsupervised stack of RNNs. At the input level, it learns to predict its next input from the previous inputs. Only unpredictable inputs of some RNN in the hierarchy become inputs to the next higher level RNN, which therefore recomputes its internal state only rarely. Each higher level RNN thus studies a compressed representation of the information in the RNN below. This is done such that the input sequence can be precisely reconstructed from the representation at the highest level. The system effectively minimizes the description length or the negative logarithm of the probability of the data. Given a lot of learnable predictability in the incoming data

sequence, the highest level RNN can use supervised learning to easily classify even deep sequences with long intervals between important events. It is possible to distill the RNN hierarchy into two RNNs: the "conscious" chunker (higher level) and the "subconscious" automatizer (lower level). Once the chunker has learned to predict and compress inputs that are unpredictable by the automatizer, then the automatizer can be forced in the next learning phase to predict or imitate through additional units the hidden units of the more slowly changing chunker. This makes it easy for the automatizer to learn appropriate, rarely changing memories across long intervals. In turn, this helps the automatizer to make many of its once unpredictable inputs predictable, such that the chunker can focus on the remaining unpredictable events. A generative model partially overcame the vanishing gradient problem of automatic differentiation or backpropagation in neural networks in 1992. In 1993, such a system solved a "Very Deep Learning" task that required more than 1000 subsequent layers in an RNN unfolded in time.

Section 7

The Hopfield network is an RNN in which all connections across layers are equally sized. It requires stationary inputs and is thus not a general RNN, as it does not process sequences of patterns. However, it guarantees that it will converge. If the connections are trained using Hebbian learning, then the Hopfield network can perform as robust content-addressable memory, resistant to connection alteration.